Nike Supply Chain Control Tower: Comprehensive Metric Intelligence

This document provides a technical audit and data lineage mapping for all metrics displayed in the Algoleap Control Tower. Every claim is traced to specific architectural components and logic within the tiered workspace.

📊 Page 1: Dashboard Overview

Purpose: Real-time visibility into global inventory health, supply disruption signals, and partner capacity.

Primary Aggregator: `core/orchestration.py:get_initial_state()`

Data Frequency: Live on-load (JSON-driven).

🏷️ Section: KPI Performance Row

1. Forecast Accuracy (MAPE)

- **UI Indicator**: `kpi-card` (Label: "Forecast accuracy (MAPE)")
- **Data Source**: `data/datasets/demand_planning_dataset.csv`
- **Data Fields Used**: `demand_planning_dataset.csv` -> `forecast_accuracy` (Float)
- **Calculation Logic**:

1. Group records by `product_name`.
2. Compute `MEAN(forecast_accuracy)`.
3. Convert to percentage: `avg_acc * 100`.
4. **Delta Logic**: Compared against a hardcoded baseline of `88.9%` (`avg_acc - 88.9`).

- **Data Flow**: `CSV` -> `orchestration.py:load_demand_forecast()` -> `orchestration.py:build_kpis()` -> `InitialState` -> `UI (renderDashboard)`.
- **Business Significance**: Measures demand visibility quality; critical for reducing safety stock over-provisioning.

2. Network Fill Rate

- **UI Indicator**: `kpi-card` (Label: "Network fill rate")
- **Data Source**: `data/Dataset/inventory_positions.csv` (or `raw_inventory.json` fallback)
- **Data Fields Used**: `on_hand_units` (Integer), `demand_forecast` (Integer)
- **Calculation Logic**:

5. Iterate through all loaded `InventoryItem` objects.
6. Filter for SKUs where `on_hand_units >= demand_forecast`.
7. Calculation: `(Count[Filled] / Count[Total]) * 100`.

- **Data Flow**: `CSV/JSON` -> `orchestration.py:load_inventory()` -> `orchestration.py:build_kpis()` -> `UI`.
- **Business Significance**: Proximal measure of customer service level; identifies lost revenue potential.

3. Total On-Hand Units

- **UI Indicator**: `kpi-card` (Label: "Total on-hand units")
- **Data Source**: `data/Dataset/inventory_positions.csv` (or `JSON` fallback)
- **Data Fields Used**: `on_hand_units` (Integer)
- **Calculation Logic**: `SUM(on_hand_units)` across the top 50 filtered SKUs.
- **Data Flow**: `CSV` -> `orchestration.py:load_inventory()` -> `orchestration.py:build_kpis()`.
- **Business Significance**: Visibility into capital tied up in inventory across the global DC network.

4. Active Supply Alerts

- **UI Indicator**: `kpi-card` (Label: "Active supply alerts")
- **Data Source**: `data/Dataset/market_signals.csv`
- **Data Fields Used**: `intensity` (Float)
- **Calculation Logic**: `COUNT(signals)` where intensity (mapped 0-10) exceeds `SIGNAL_THRESHOLD` (7.0).
- **Data Flow**: `CSV` -> `orchestration.py:load_signals()` -> `orchestration.py:build_kpis()`.
- **Business Significance**: Proactive risk detection; counts high-intensity events requiring scenario intervention.

🏛️ Section: DC Network Health Grid

DC Cluster Capacity (%)

- **UI Component**: `dc-card` (Grid of 6)
- **Data Source**: `data/Dataset/raw/raw_inventory.json` (Primary Source)
- **Data Fields Used**: `on_hand_units`, `safety_stock_level`, `distribution_center`
- **Calculation Logic**:

8. Aggregate `on_hand` and `safety_stock` by DC name.
9. Establish **Target Capacity**: `safety_stock * 2.5` (Operational Heuristic).
10. Calculation: `(Current_Units / Target_Capacity) * 85` (Scaled to UI bar).
11. **Status Class**:

- `OVERSTOCK` (Amber): > 90%
- `CRITICAL` (Red): < 50%
- `STABLE` (Green): 50% - 90%

- **Data Flow**: `JSON` -> `orchestration.py:load_dc_network()` -> `DCCard` -> `UI`.
- **Business Significance**: Identifies regional bottlenecks and over-utilization of physical warehouse nodes.

🌐 Section: Partner & Market Analytics

Demand Planning (MAPE Distribution)

- **UI Component**: `forecast-bars`
- **Data Source**: `data/datasets/demand_planning_dataset.csv`
- **Calculation**: `MEAN(forecast_accuracy)` per SKU.
- **Logic**: Top 7 SKUs by accuracy are selected. Bar colors are conditional: Green (>=95%), Blue (>=91%), Amber (<91%).

Supplier Capacity Analytics

- **UI Component**: `sup-table`
- **Data Source**: `data/datasets/supplier_dataset.csv`
- **Data Fields Used**: `current_utilization` (Float), `lead_time_days` (Mean + StdDev).

- **Logic**: `utilization > 0.92` -> DISRUPTED; `> 0.85` -> HIGH UTIL; else AVAILABLE.
- **Business Significance**: Maps tier-1 dependency health; identifies production lock-outs.

🛠️ Page 2: Scenario Analysis Page

Purpose: AI-driven evaluation of logistics trade-offs for triggered disruptions.

Primary Logic: `agents/simulation.py` and `agents/evaluation.py`.

📊 Metric: Scenario "Why?" (Strategic Rationale)

- **Data Source**: Dynamic Generator in `orchestration.py:get_dynamic_scenario()`.
- **Logic**: Uses detected signal intensity or inventory gap to explain the trigger.

- *Example (Demand)*: `why = (intensity > 4.0) -> Trigger response to capture revenue uplift.`

- **Business Significance**: Provides context for the "Options Found" pill.

🎲 Metric: Monte Carlo Probability (Score Distribution)

- **Component**: `Scenario Card` -> `Processing Logs`
- **Source**: `SimulationAgent.simulate()`
- **Calculation Logic**:

12. Runs **100 iterations** of the `RandomForestRegressor`.
13. Introduces **Stochastic Jitters**: 4% cost variance, 1.2 day lead-time variance, 3% risk jitter.
14. Outputs: `monte_min`, `monte_max`, `monte_avg`, `monte_std`.

- **Business Significance**: Stress-tests AI recommendations against real-world stochasticity.

📉 Metric: Regret Index (Opportunity Cost)

- **Source**: `agents/evaluation.py:StatisticalAgent.analyze_regret()`
- **Calculation Logic**:

15. Calculates the "Global Max" score across all generated scenarios.

16. Formula: `Regret = (MAX_Score - YOUR_Score) / MAX_Score`.

- **Business Significance**: Identifies the cost of not choosing the most optimal path; measures decision efficiency.

🔗 Cross-Component Insights & Lineage

🛡️ Shared Source: `inventory_positions.csv`

This file serves as the ground truth for **Network Fill Rate**, **Total On-Hand**, **DC Distribution**, and **SKU Status Table**. Discrepancies here affect 70% of the dashboard metrics.

🧩 Data Lineage Summary

`mermaid

graph LR

A["Raw Data (CSV/JSON)"] --> B["Ingestion Agents (NLP/Cleaning)"]

B --> C["Orchestration (Aggregation/Heuristics)"]

C --> D["Simulation/KPI Agents (ML/RF)"]

D --> E["UI Framework (JS/DOM Mapping)"]

E --> F["Operational KPI (MAPE, Fill Rate, Regret)"]

`

🔍 Missing Logic / Insufficient Evidence

- **Carrier Selection**: Currently uses static naming convention (e.g., "DHL Nike Fast-Track"); no carrier-level performance API found in the current datasets.
- **Environmental Policy**: The `CO2_uplift` is a stochastic multiplier (`round(random.uniform(0.5, 3.0), 1)`); no verified carbon emissions database found in the filesystem.